p2p_sdr_algorithm

P2P Energy Sharing Algorithm with SDR-based Pricing

1. Overview

This algorithm implements a Peer-to-Peer (P2P) energy sharing mechanism for a microgrid of prosumers (consumers with PV generation). The core concept is to determine internal energy trading prices dynamically based on the **Supply-Demand Ratio (SDR)** within the community.

The system operates on a **Distributed Iterative Approach** where an Energy Sharing Provider (ESP) acts as a coordinator (or smart contract) to update prices, while prosumers locally optimize their flexible load consumption to minimize individual costs.

Key System Parameters

- **Sampling Time (Δt):** 15 minutes
- **Time Horizon (H):** N_{slots} (e.g., 96 slots for a 24-hour day-ahead schedule)
- **Set of Prosumers ($\mathcal{N}$):** {1, 2, ..., n}

---

2. Definitions and Terms

2.1 Variables per Prosumer i at time h

- P_i^h: Forecasted PV power generation (kW).
- $L_{i,base}^h$: Baseline/Inflexible load demand (kW).
- x_i^h: **Decision Variable**. Adjusted total power consumption (kW).
- $TP_i^{ref,h}$: Reference (original) total power consumption profile.
- NP_i^h: Net Power position.
  - $NP_i^h = x_i^h - P_i^h$
  - If $NP_i^h > 0$: Prosumer is a **Buyer**.
  - If $NP_i^h < 0$: Prosumer is a **Seller**.

2.2 Community Aggregates

- **TSP (Total Selling Power):** Sum of energy surplus from all selling prosumers.

$$TSP^h = \sum_{i \in \mathcal{N}, NP_i^h < 0} |NP_i^h|$$

- **TBP (Total Buying Power):** Sum of energy deficit from all buying prosumers.

$$TBP^h = \sum_{i \in \mathcal{N}, NP_i^h \geq 0} NP_i^h$$

- **SDR (Supply-Demand Ratio):**

$$SDR^h = \frac{TSP^h}{TBP^h}$$

2.3 Pricing Parameters

- λ_{grid}^{buy}: Price to buy electricity from the main utility grid (High price).
- λ_{grid}^{sell}: Feed-in tariff price to sell to the utility grid (Low price).
- $Pr_{internal}^{buy,h}, Pr_{internal}^{sell,h}$: Dynamic internal P2P trading prices.

---

3. Mathematical Models

3.1 Objective Function (Prosumer Side)

Each prosumer i aims to minimize their total cost, which is a sum of the **Economic Cost** and the **Inconvenience Cost** (cost of shifting load).

$$\min_{x_i} C_i = \sum_{h=1}^{H} \left[\text{Cost}_{elec}(x_i^h) + \text{Cost}_{inconv}(x_i^h) \right]$$

Economic Cost Term:

$$\text{Cost}_{elec}(x_i^h) = \begin{cases} Pr_{internal}^{buy,h} \cdot NP_i^h & \text{if } NP_i^h \geq 0 \text{ (Buying)} \\ Pr_{internal}^{sell,h} \cdot NP_i^h & \text{if } NP_i^h < 0 \text{ (Selling)} \end{cases}$$

Inconvenience Cost Term:
Modeled as a quadratic function representing user discomfort when deviating from their preferred schedule (TP_i^{ref}). α_i is a sensitivity coefficient.

$$\text{Cost}_{inconv}(x_i^h) = \alpha_i \left(x_i^h - TP_i^{ref,h} \right)^2$$

3.2 Pricing Mechanism (ESP Side)

The internal prices are bounded between grid prices [$\lambda_{grid}^{sell}, \lambda_{grid}^{buy}$] and depend on the SDR.

Case A: Supply Shortage ($0 \leq SDR^h \leq 1$)

- Local generation is less than local demand.
- Sellers are incentivized with higher prices.
- Buyers pay a weighted average of internal generation and grid imports.

$$Pr_{internal}^{sell,h} = \frac{\lambda_{grid}^{buy} \cdot \lambda_{grid}^{sell}}{(\lambda_{grid}^{buy} - \lambda_{grid}^{sell}) \cdot SDR^h + \lambda_{grid}^{sell}}$$

$$Pr_{internal}^{buy,h} = Pr_{internal}^{sell,h} \cdot SDR^h + \lambda_{grid}^{buy} \cdot (1 - SDR^h)$$

Case B: Supply Surplus ($SDR^h > 1$)

- Local generation exceeds local demand.
- Excess energy must be exported to the grid.
- Prices collapse to the grid feed-in tariff to discourage over-generation or encourage consumption.

$$Pr_{internal}^{sell,h} = \lambda_{grid}^{sell}$$

$$Pr_{internal}^{buy,h} = \lambda_{grid}^{sell}$$

(Note: A small service fee β can be added to buy prices if required by the business model).

---

4. Pseudocode

```
ALGORITHM: P2P_SDR_Energy_Sharing
INPUTS:
    H           = 96              # 15-min intervals for 24 hours
    N_prosumers = [...]           # List of prosumer objects
    Lambda_buy  = 0.20            # Grid buy price ($/kWh)
    Lambda_sell = 0.08            # Grid sell price ($/kWh)
    Epsilon     = 1e-4            # Convergence threshold
    Max_Iter    = 100             # Safety break

INITIALIZATION:
    # ESP broadcasts initial internal prices (e.g., mid-point of grid
prices)
    For h in 1 to H:
        Pr_sell[h] = (Lambda_buy + Lambda_sell) / 2
        Pr_buy[h]  = (Lambda_buy + Lambda_sell) / 2
```

```
MAIN LOOP (Distributed Iteration):
    For k = 1 to Max_Iter:

        # --- STEP 1: Local Optimization (Prosumers) ---
        # (This step happens in parallel for each prosumer)
        For each prosumer i in N_prosumers:
            # Minimize Cost: Electricity Cost + Alpha * (Current_Load -
Ref_Load)^2
            # Subject to:
            #   1. Total Energy Constraint: Sum(x_i^h) == Sum(Ref_Load^h)
            #   2. Min/Max Load Constraints per slot

            x_i = Minimize_Cost(Prices=Pr_buy, Pr_sell, Alpha_i, PV_i,
Load_Ref_i)

            # Calculate Net Power
            NP_i[h] = x_i[h] - PV_i[h]
            Send NP_i to ESP

        # --- STEP 2: Aggregation (ESP) ---
        Previous_Pr_sell = Pr_sell.copy()

        For h in 1 to H:
            TSP[h] = 0
            TBP[h] = 0

            # Calculate Total Selling and Buying Power
            For each prosumer i:
                If NP_i[h] < 0:
                    TSP[h] += abs(NP_i[h])
                Else:
                    TBP[h] += NP_i[h]

            # Calculate SDR (handle division by zero)
            If TBP[h] == 0:
                SDR[h] = infinity # Surplus case
            Else:
                SDR[h] = TSP[h] / TBP[h]

            # --- STEP 3: Pricing Update (ESP) ---
            If SDR[h] >= 1:
                # Surplus Scenario
                Pr_sell[h] = Lambda_sell
                Pr_buy[h]  = Lambda_sell

            Else (0 <= SDR[h] < 1):
```

```
                # Shortage Scenario
                # Inverse proportional mapping
                Pr_sell[h] = (Lambda_buy * Lambda_sell) / \
                             ((Lambda_buy - Lambda_sell) * SDR[h] +
Lambda_sell)

                # Buyer price balances the cost of internal purchase + grid
import
                Pr_buy[h]  = Pr_sell[h] * SDR[h] + Lambda_buy * (1 - SDR[h])

        # --- STEP 4: Convergence Check ---
        Price_Deviation = Norm(Pr_sell - Previous_Pr_sell)

        If Price_Deviation < Epsilon:
            Break Loop (Convergence Reached)

OUTPUT:
    Optimal Load Profiles x_i for all i
    Final Internal Clearing Prices Pr_sell, Pr_buy
    Financial Settlement details
```